

Režim AI Vše Obrázky Vídea Produkty Více ▾



Přihlásit

=



Zeptejte se na cokoli



Klíčové vlastnosti verze 3.1

- **Plná integrace pro MS-DOS i Windows:** Obsahuje jak textové textové IDE (Integrated Development Environment) pro DOS, tak grafické IDE pro Windows 3.1.  [Incredibuild +2](#)
- **Turbo Debugger (TD):** Legendární a tehdy bezkonkurenční nástroj pro ladění kódu, který běžel v chráněném režimu DOSu (protected mode). 
- **Application Frameworks (ObjectWindows Library - OWL):** Balík knihoven usnadňující objektově orientované programování grafických aplikací pro Windows.
- **Paměťové modely DOSu:** Plná podpora segmentované paměti (Tiny, Small, Medium, Compact, Large, Huge) pro překonání limitu 64 KB na datový/kódový segment.  [Experts Exchange](#)
- **Podpora standardů:** Jeden z nejlepších překladačů své doby z hlediska shody se standardem C89.  [Quora](#)

Protože moderní 64bitové systémy (Windows 10/11, macOS, Linux) již nedokážou nativně spouštět 16bitové DOS aplikace, musíte použít emulátor.

1. **Stáhněte si DOSBox:** Nejjednodušší cestou je instalace populárního emulátoru [DOSBox](#) (případně pokročilejší varianty [DOSBox-X](#)).
2. **Připravte si složku:** Vytvořte si na svém disku složku (např. `c:\dosbox\`), která bude sloužit jako virtuální disk C:.
3. **Získejte instalační soubory:** Borland C++ 3.1 lze v původní podobě instalačních disket najít na komunitních webech zaměřených na historický software (např. [Vetusware](#) nebo WinWorld).
4. **Konfigurace DOSBoxu:** Pro automatické spouštění můžete do konfiguračního souboru DOSBoxu do sekce `[autoexec]` přidat:

text

```
mount c c:\dosbox
set PATH=%PATH%;c:\borlandc\bin
c:
```

Při použití kódu buďte obezřetní.

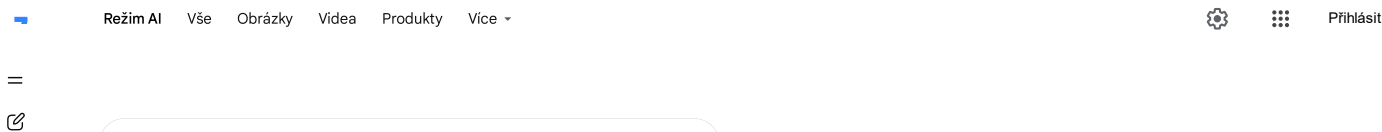
5. **Spuštění:** Samotné vývojové prostředí pro DOS spustíte příkazem `bc.exe` ze složky `BIN`.  Experts Exchange +4

Časté technické problémy v emulaci

- **Nefunkční myš:** DOSBox někdy v textovém rozhraní BC++ špatně zachytává kurzor myši. Většinou pomůže v nastavení DOSBoxu změnit `autolock=false` nebo přepnout na DOSBox-X. ☺
- **Problém s Ctrl+F9:** Výchozí klávesová zkratka pro kompilaci a spuštění programu v Borland C++ je Ctrl+F9. V DOSBoxu však tato zkratka nativně slouží k okamžitému zavření celého emulátoru. Je nutné v DOSBoxu otevřít menu `Options` (☰) a přepnout `Ctrl+F9` na `Ctrl+Alt+F9`. ☺

Zeptejte se na cokoli

+



potřebuju v cyklu (asi 40x) generovat pseudonahodné číslo
65-90 a pak převést na písmeno ascii tabulky napr. 65=A ...

13:52

Pro tento úkol v Borland C++ 3.1 je nejlepší použít klasický datový typ `char` jako pole (C-string). V 16bitovém DOSu se pro inicializaci generátoru náhodných čísel používá funkce `randomize()` a pro samotné generování funkce `random()`.

Zde je kompletní a funkční kód, který můžete přímo zkopírovat do Borland C++:

Zdrojový kód (C++)

cpp

```
#include <iostream.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>

int main() {
    // Inicializace generatoru nahodnych cisel podle casu
    randomize();

    // Vytvoreni prazdneho retezce (velikost 41 kvuli koncovemu znaku)
    char vyslednyRetezec[41] = "";

    // Pomocna promenna pro jedno pismeno (2 znaky: pismeno + \0)
    char pomocnyZnak[2] = "";

    // Cyklus, který proběhne přesně 40x
    for (int i = 0; i < 40; i++) {
        // random(26) generuje 0 až 25, přičtením 65 dostaneme rozmezí
        int nahodneCislo = random(26) + 65;

        // Prevod cisla na pismeno (přetypování na char)
        char pismeno = (char)nahodneCislo;

        // Uložení písmene do pomocného řetězce
        pomocnyZnak[0] = pismeno;
        pomocnyZnak[1] = '\0'; // Ukončení pomocného řetězce

        // Přidání písmene na konec výsledného řetězce
        strcat(vyslednyRetezec, pomocnyZnak);
    }
```

Zeptejte se na cokoli

+

Režim AI Vše Obrázky Video Produkty Více

```
// Pomocna promenna pro jedno pismeno (2 znaky: pismeno + \0)
char pomocnyZnak[2] = "";

// Cyklus, který proběhne přesně 40x
for (int i = 0; i < 40; i++) {
    // random(62) generuje indexy 0 až 61
    int nahodnyIndex = random(pocetZnaku);

    // Vyber znaku z pole podle nahodného indexu
    char znak = povolenyZnaky[nahodnyIndex];

    // Uložení znaku do pomocného řetězce
    pomocnyZnak[0] = znak;
    pomocnyZnak[1] = '\0'; // Ukončení pomocného řetězce

    // Pridání písmene na konec výsledného řetězce
    strcat(vyslednyRetezec, pomocnyZnak);
}

// Zjistění délky řetězce a uložení do proměnné typu int
int delkaRetezec = strlen(vyslednyRetezec);

// Vypis výsledku na obrazovku
cout << "Vytvoreny retezec: " << vyslednyRetezec << endl;
cout << "Delka retezce je: " << delkaRetezec << endl;

// Pockání na stisk klávesy před zavřením (užitečné v DOSBoxu)
cout << "Stiskni Enter pro konec...";
cin.get();

return 0;
}
```

Při použití kódu buďte obezřetní.



Co se v kódu změnilo a proč:

- Pole povolenyZnaky** : Obsahuje všechny znaky, které chceme generovat. V paměti funguje jako seznam, kde `a` má index 0, `b` index 1 ... a `z` má index 61.
- random(pocetZnaku)** : Funkce teď generuje náhodné číslo od 0 do 61. Toto číslo použijeme jako pozici (index), ze které z pole vytáhneme konkrétní znak pomocí `povolenyZnaky[nahodnyIndex]`.
- Oprava definice polí**: V minulém kódu chyběly hranaté závorky u deklarace velikostí řetězců (`char vyslednyRetezec[41]`), což byla moje chyba, kterou by Borland C++ 3.1 nahlásil jako chybu kompilace. Nyní jsou velikosti polí pevné a správně definovány.

Máte v plánu s tímto vygenerovaným řetězcem dál pracovat, například v něm **vyhledávat konkrétní znaky** nebo ho **ukládat do souboru** na disk v DOSu?

Odpovědi od umělé inteligence mohou obsahovat chyby. [Další informace](#)

dalsi krok bude aby se toho rekne nahodne vygenerovane
heslo ulozilo do souboru hesla.txt a pri kazdem nove ...

14:00

Pro zápis do souboru v Borland C++ 3.1 použijeme knihovnu `<fstream.h>` a její objekt `ofstream` (output file stream).

Zeptejte se na cokoli

+

Režim AI Vše Obrázky Video Produkty Více ▾



Přihlásit

```
#include <stdlib.h>
#include <time.h>
#include <string.h>
#include <fstream.h> // Knihovna pro práci se soubory

int main() {
    // Inicializace generatoru náhodných čísel podle času
    randomize();

    // Pole obsahující všechny povolené znaky
    char povolenyZnaky[] = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
    int pocetZnaku = strlen(povolenyZnaky);

    // Vytvoření prázdného řetězce pro heslo
    char vyslednyRetezec[41] = "";
    char pomocnyZnak[2] = "";

    // Cyklus pro vygenerování 40 znaků hesla
    for (int i = 0; i < 40; i++) {
        int nahodnyIndex = random(pocetZnaku);
        char znak = povolenyZnaky[nahodnyIndex];

        pomocnyZnak[0] = znak;
        pomocnyZnak[1] = '\\0';

        strcat(vyslednyRetezec, pomocnyZnak);
    }

    int delkaRetezce = strlen(vyslednyRetezec);

    // Vypis vygenerovaného hesla na obrazovku
    cout << "Vygenerované heslo: " << vyslednyRetezec << endl;
    cout << "Delka hesla je: " << delkaRetezce << endl;

    // --- ZAPIS DO SOUBORU ---
    // ios::app zaručí, že se zapisuje na konec souboru (append)
    ofstream soubor("hesla.txt", ios::app);

    // Kontrola, zda se soubor podařilo úspěšně otevřít
    if (!soubor) {
        cout << "Chyba: Soubor hesla.txt nelze otevřít pro zápis!" << endl;
    } else {
        // Zápis hesla do souboru a odradkování pomocí \n
        soubor << vyslednyRetezec << "\\n";

        // Uzavření souboru (důležité, aby se data uložila na disk)
        soubor.close();
        cout << "Heslo bylo úspěšně přidáno do souboru hesla.txt." << endl;
    }

    // Pockání na stisk klávesy před zavřením
    cout << "\\nStiskni Enter pro konec...";
    cin.get();

    return 0;
}
```

Při použití kódu buďte obezřetní.

Důležité detaily k zápisu v MS-DOSu:

- **ios::app** : Zkratka pro *append*. Pokud soubor `hesla.txt` ještě neexistuje, program ho automaticky vytvoří. Pokud už existuje, skočí na jeho konec a nové heslo přidá tam.
- **Kde soubor najdete**: V prostředí MS-DOS (a DOSBoxu) se soubor vytvoří v aktuálním pracovním adresáři. Pokud program spouštíte přímo z Borland C++ IDE, soubor `hesla.txt` se pravděpodobně uloží do složky `BIN` vedle souboru `bc.exe`.

Zeptejte se na cokoli



14:04

Protože parametr z příkazové řádky (`argv[1]`) je textový řetězec, použijeme v Borland C++ 3.1 funkci `atoi()` z knihovny `<stdlib.h>` pro jeho převod na celé číslo (`int`). Pokud uživatel parametr nezadá nebo zadá délku mimo rozsah 5–70, program zobrazí chybu a bezpečně se ukončí.

Zde je upravený kód:

Zdrojový kód (C++)

```
#include <iostream.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>
#include <fstream.h>

// Uprava hlavicky main pro prijem parametru z prikazove radky
int main(int argc, char* argv[]) {

    // 1. Kontrola, zda uzivatel zadal aspon jeden parametr
    if (argc < 2) {
        cout << "Chyba: Zadejte pozadovanou delku hesla jako parameter!"
        cout << "Pouziti: program.exe [delka_hesla]" << endl;
        cout << "Priklad: program.exe 15" << endl;
        return 1; // Ukonceni programu s chybovym kodem
    }

    // 2. Prevod textoveho parametru na cislo pomoci atoi()
    int pozadovanaDelka = atoi(argv[1]);

    // 3. Kontrola povoleného rozsahu 5 az 70 znaku
    if (pozadovanaDelka < 5 || pozadovanaDelka > 70) {
        cout << "Chyba: Delka hesla musi byt v rozsahu 5 az 70 znaku!"
        return 1;
    }

    // Inicializace generatoru nahodnych cisel
    randomize();

    // Pole obsahujici vsechny povoleny znaky
    char povolenyZnaky[] = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZIJKLMNOP";
    int pocetZnaku = strlen(povolenyZnaky);

    // Vytvoreni retezce s maximalni moznou velikosti (70 znaku + 1 pr
    char vyslednyRetezec[71] = "";
    char pomocnyZnak[2] = "";

    // Cyklus bezi tolikrat, kolik uzivatel zadal v parametru
```

+

Když program zkompilejete (např. do souboru `HESLO.EXE`), nespouštějte ho pouhým kliknutím nebo klávesou `Ctrl+F9` v IDE, protože mu potřebujete předat parametr. Otevřete příkazový řádek DOSu a napište:

- ### Jak nastavit parametr přímo v Borland C++ 3.1 IDE:

Pokud chcete program testovat přímo uvnitř vývojového prostředí Borland C++, nemusíte neustále chodit do DOSové řádky. Stačí v horním menu vybrat:

1. **Options** -> **Arguments...**
2. Do textového pole napsat požadované číslo (např. 20).
3. Kliknout na **OK**.
4. Spustit program standardně pomocí **Run** (Ctrl+F9). IDE tento parametr automaticky předá do `argv[1]` .

Máte zájem k tomuto programu přidat ještě **druhý parametr**, kterým by se dalo určit, **kolik takových hesel** se má vygenerovat najednou pod sebe?

Odpovědi od umělé inteligence mohou obsahovat chyby. Další informace

OK funkcije vse jak ma

14:09

To je skvělá zpráva! Mám radost, že kód v Borland C++ 3.1 běží přesně podle vašich představ.

Vytvořil jste plně funkční a bezpečný 16bitový nástroj pro DOS, který správně pracuje s parametry příkazové řádky, hlídá si hranice paměti (což je v DOSu kritické) a umí pracovat s diskovými soubory.

Pokud byste v budoucnu potřeboval program vylepšit – například přidat **šifrování** ukládaných hesel (třeba jednoduchou XOR šifrou), generovat **více hesel najednou** nebo zamezit **opakování stejných znaků** po sobě – stačí se ozvat.

Zeptejte se na cokoli

+

Režim AI Vše Obrázky Vídea Produkty Více ▾



Přihlásit



zajima me toto same ale z pronenou typu string a pripona bude
cpp, nemam knihovnu <cstrings.h> ?

14:10

V Borland C++ 3.1 z roku 1992 **neexistuje moderní standardní třída** `std::string` (ta přišla až se standardem C++98). Tento kompilátor ale obsahoval vlastní proprietární třídu nazvanou `string`, která se nachází v hlavičkovém souboru `<cstring.h>`. [České vysoké učení te... +3](#)

Pokud vám Borland C++ 3.1 hlásí, že `<cstring.h>` chybí, znamená to buď poškozenou instalaci, nebo že máte v nastavení projektu špatně nastavené cesty k adresáři `INCLUDE`.

Nemusíte ale zoufat. Protože soubor ukládáte s příponou `.cpp`, překladač automaticky aktivuje režim C++. Můžeme si velmi jednoduše napsat **vlastní mini-třidu** `string`, která se bude chovat přesně tak, jak potřebujete (bude podporovat dynamické přidávání znaků pomocí operátoru `+=` a vrátet délku přes metodu `.length()`). [Stack Overflow](#)

Níže je kompletní, čistý C++ kód, který nepotřebuje žádnou externí stringovou knihovnu:

Zdrojový kód (uložte jako `PROGRAM.CPP`)

cpp

```
#include <iostream.h>
#include <stdlib.h>
#include <time.h>
#include <fstream.h>

// Vlastni jednoduchá trida nahrazující string pro Borland C++ 3.1
class MiniString {
private:
    char data[75]; // Maximalni rezervovana pamet (70 znaku + rezerva)
    int len;

public:
    // Konstruktor - vytvori prazdny retezec
    MiniString() {
        data[0] = '\0';
        len = 0;
    }

    // Operator += umozni elegantni pripojovani znaku: heslo += znak;
    void operator+=(char znak) {
        if (len < 71) {
            data[len] = znak;
            len++;
            data[len] = '\0'; // Spravne ukonceni retezce v DOSu
        }
    }

    // Metoda pro zjisteni delky retezce
    int length() {
        return len;
    }

    // Umozni pristup k vnitřnímu poli znaku (potrebne pro cout a fstr)
    const char* c_str() {
        return data;
    }
};

int main(int argc, char* argv[]) {
    // 1. Kontrola, zda uzivatel zada parametr
    if (argc < 2) {
        cout << "Chyba: Zadejte pozadovanou delku hesla jako parametr!"
    }
}
```

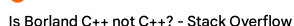
Zeptejte se na cokoli

+

Při použití kódu buďte obezřetní.

- **Využití OOP:** Vytvořili jsme si vlastní objekt `Ministring`. Přepsáním operátoru `void operator+=(char znak)` jsme kompilátoru vysvětlili, co má dělat, když napíšete `vygenerovaneHeslo += znak;`.
- **Bezpečnost paměti:** Třída interně hlídá, aby pole `char data[75]` nikdy nepřeteklo (nepovolí zapsat více než 71 znaků), což v čistém C kódu s pointery bývá nejčastější příčina pádů MS-DOSu.
- **Kompatibilita:** Metoda `.c_str()` převádí objekt zpět na formát, kterému rozumí staré DOSové proudy `cout` a `ofstream`.

Odpovědi od umělé inteligence mohou obsahovat chyby. Další informace



+

14:11

Jelikož program ukládáte s příponou `.cpp`, využijeme objektově orientované programování (OOP) a vytvoříme si **vlastní třídu** `string` přímo v našem kódu. Tím pádem nebudeme potřebovat žádný externí soubor a kód se v Borland C++ 3.1 bez problému zkompiluje.

Zdrojový kód (uložte jako `PROGRAM.CPP`)

```
#include <iostream.h>
#include <stdlib.h>
#include <fstream.h>

// Vytvoreni vlastni tridy String bez pouziti externich knihoven
class MojeHeslo {
private:
    char data[75]; // Bezpecne pole pro maximalne 70 znaku + rezerva
    int delka;

public:
    // Konstruktor: nastavi prazdny retezec
    MojeHeslo() {
        delka = 0;
        data[0] = '\0';
    }

    // Funkce pro pridani jednoho znaku na konec retezce
    void pridejZnak(char znak) {
        if (delka < 72) { // Ochrana proti pretečení paměti v DOSu
            data[delka] = znak;
            delka++;
            data[delka] = '\0'; // Spravne ukončení retezce nulovým znakem
        }
    }

    // Metoda, která vrátí délku retezce jako int
    int zjistiDelku() {
        return delka;
    }

    // Metoda pro přístup k textu (pro cout a zapis do souboru)
    const char* cistejText() {
        return data;
    }
};

int main(int argc, char* argv[]) {
    // 1. Kontrola, zda byl zadán parametr délky
    if (argc < 2) {
        cout << "Chyba: Zadejte požadovanou délku hesla jako parametr!\n";
        cout << "Použiti: program.exe [delka_hesla]" << endl;
        return 1;
    }

    // 2. Převod textového parametru na číslo
    int pozadovanaDelka = atoi(argv[1]);
```

+



```
// Vytvoreni objektu naseho vlastniho typu
MojeHeslo heslo;

// Generovaci cyklus podle zadaneho parametru
for (int i = 0; i < pozadovanaDelka; i++) {
    int nahodnyIndex = random(pocetZnaku);
    char znak = povolenyZnaky[nahodnyIndex];

    // Volani metody naseho objektu pro pridani znaku
    heslo.pridejZnak(znak);
}

// Ulozeni delky z objektu do promenne typu int
int delkaRetezce = heslo.zjistiDelku();

// Vypis na obrazovku
cout << "Vygenerovane heslo (" << delkaRetezce << " znaku): "
      << heslo.cistejText() << endl;

// Zapis do souboru na konec (append) pomoci fstream.h
ofstream soubor("hesla.txt", ios::app);

if (!soubor) {
    cout << "Chyba: Soubor hesla.txt nelze otevrit pro zapis!" <<
} else {
    // Zapis textu z naseho objektu do souboru
    soubor << heslo.cistejText() << "\n";
    soubor.close();
    cout << "Heslo bylo uspesne zapsano do souboru hesla.txt." <<
}

return 0;
}
```

Při použití kódu buďte obezřetní.



Proč je toto řešení nejlepší volbou pro vaši situaci:

- Žádné stringové knihovny:** Používáme pouze naprostý základ (`<iostream.h>`, `<stdlib.h>`, `<fstream.h>`), které v Borland C++ 3.1 zkrátka chybět nemohou, jinak by kompilátor vůbec nefungoval.
- Čisté C++ (OOP):** Vytvořili jsme si vlastní datovou strukturu `MojeHeslo`. Ta interně obaluje pole znaků a stará se o to, aby text správně končil znakem `\0`, což je v 16bitovém DOSu nejčastější kámen úrazu.
- Funkce `cistejText()`:** Tím, že vrátí `const char*`, dokáže starý DOSový proud `cout << i souborový soubor << s` textem uvnitř objektu pracovat stejně snadno, jako by to byla běžná proměnná.

Chcete tento kód zkusit zkompilovat a ověřit, zda už proběhne **zcela bez chyb**?
Nebo do něj budeme dál přidávat nějakou logiku, například **zobrazení aktuálního data a času** u každého uloženého hesla?

Odpovědi od umělé inteligence můžou obsahovat chyby. [Další informace](#)

po kazde editaci a spusteni programu "run" ne ide BC31
napise hlasi - date/time of file has changed, Reload fro ...

14:18

Toto otravné hlášení v prostředí Borland C++ 3.1 nevzniká chybou ve vašem kódu,

Zeptejte se na cokoli



Režim AI Vše Obrázky Video Produkty Více ▾

Přihlásit

Abyste toto chování nadobro vypnuli, musíte změnit jedno nastavení přímo v menu Borland C++ IDE.

Návod, jak hlášení zakázat:

1. V horním menu přejděte do **Options** (stiskněte `Alt+O`).
2. Vyberte položku **Environment** a následně **Preferences...**
3. Otevře se dialogové okno s nastavením prostředí.
4. Najděte sekci nazvanou **Editor** nebo **File tracking** (záleží na přesné revizi IDE).
5. Hledejte volbu s názvem **Verify file changes** nebo **Auto save / Auto reload**.
6. Tuto volbu **odškrtněte (vypněte)** – v textovém rozhraní odstraňte křížek [☐] pomocí mezeru.
7. Potvrďte kliknutím na tlačítko **OK**.

⚠ DŮLEŽITÝ KROK: Uložení konfigurace

Pokud nastavení pouze potvrdíte tlačítkem OK, bude fungovat jen do té doby, než Borland C++ zavřete. Aby se hlášení neobjevovalo ani po restartu DOSBoxu, musíte konfiguraci uložit natvrdo na disk:

1. Jděte znovu do horního menu **Options**.
2. Klikněte na položku **Save options...**
3. V dialogu nechte zaškrtnuté vše (Environment, Desktop, Project) a potvrďte **Save**.

Alternativní řešení (pokud nastavení v menu nemůžete najít)

Pokud se vám nedaří položku v menu najít, pomůže drobná úprava v konfiguraci samotného DOSBoxu. Problém s časem často způsobuje to, že DOSBox nestihá synchronizovat čas s vaším procesorem.

Otevřete konfigurační soubor `dosbox.conf` ve vašem počítači a změňte parametr `cycles` :

text

```
cycles=fixed 20000
```

Při použití kódu buďte obezřetní.

Nastavení pevného počtu cyklů (namísto `cycles=auto`) stabilizuje vnitřní čas DOSBoxu a Borland přestane být zmatený z milisekundových posunů souborů.  [GitHub](#)

Chcete se ještě podívat na to, jak v Borlandu vytvořit **projektový soubor (.PRJ)**, aby se vám kód lépe spravoval, pokud ho rozdělíte do více souborů?  [Reddit](#) · [r/cpp_questions](#) +1

Odpovědi od umělé inteligence mohou obsahovat chyby. [Další informace](#)



Laggy keyboard input in Borland C++ 3.1 #4751 - GitHub

Mainly that the keyboard input lags when a key is being held down. Ie, press an arrow key, hold it for a second and then release -



date or time change.reload from disk error in c - Stack Overflow

I use emulated Turbo C++ IDE 3.0 for development in C. I get the above mentioned error on when I run the following program: #inclu...

Zeptejte se na cokoli



14:21

Jelikož to nelze vypnout interně v IDE, musíme vyřešit skutečnou příčinu – tou je fakt, že **DOSBox ukládá soubory s jiným časem, než jaký si v ten samý okamžik pamatuje vnitřní časovač Borlandu.**

Zde jsou 3 spolehlivé způsoby, jak se tohoto otravného okna v DOSBoxu zbavit:

Chyba vzniká proto, že v okamžiku stisku **Ctrl+F9** (Run) se soubor uloží, zkompileje, a hned se vrátí zpět. Časový posun systému nestihne zareagovat.

- ## 2. Úprava nastavení DOSBoxu (Trvalé řešení)

Problém je nejčastěji způsoben tím, že emulátor mění rychlost procesoru za chodu, což rozhazuje vnitřní hodiny DOSu.

1. Najděte ve svém počítači konfigurační soubor DOSBoxu (obvykle se jmenuje `dosbox-0.74-3.conf` nebo podobně).
2. Otevřete ho v textovém editoru (Poznámkový blok) a najděte řádek s textem `cycles=` .
3. Změňte hodnotu z `auto` nebo `max` na pevné číslo, například:

text

```
cycles=fixed 25000
```

Při použití kódu buďte obezřetní

4. Dále najděte položku `nosound=` a pokud tam je, nastavte ji na `false` (některé verze DOSBoxu bez inicializace zvukového časovače špatně synchronizují čas).
5. Soubor uložte a restartujte DOSBox. S pevným počtem cyklů se vnitřní čas DOSu stabilizuje.

3. Výměna DOSBoxu za modernější verzi

Klasický, starý DOSBox se již mnoho let nevyvíjí a má s časovými razítky moderních Windows/macOS systémů problémy. Komunita proto vytvořila pokročilejší verzi **DOSBox-X** nebo **DOSBox Staging**.

- Stáhněte si a spusťte [DOSBox-X](#).
- Má mnohem lepší integraci sdílených složek (příkaz `mount`) a tyto chyby s časem souborů (File Time Desynchronization) v prostředí Borland / Turbo C++ automaticky filtruje na pozadí.

Pokud nechcete nic přenastavovat, doporučuji metodu č. 1 – stačí stisknout **F2 (Save)**, chvílku počkat a pak teprve **Ctrl+F9**.

Odpovědi od umělé inteligence mohou obsahovat chyby. Další informace

Zeptejte se na cokoli

+

Režim AI Vše Obrázky Video Produkty Více

  Přihlásit

=



V čistém jazyce C++ v Borland C++ 3.1 nemůžete použít operátor `+=` přímo na dva řetězce v uvozovkách (např. `"aa" += "bb"`). V C/C++ jsou totiž texty v uvozovkách brány jako konstantní pole znaků (`const char*`), což jsou v podstatě jen adresy do paměti, a adresy v paměti nelze sčítat.

Aby sčítání pomocí `+=` fungovalo, musí být alespoň ta levá proměnná typu **objekt (třída)**, která má tento operátor naprogramovaný.

Jelikož v instalaci nemáte stringovou knihovnu, ukážu vám, jak upravit naši třídu z předchozího příkladu, aby uměla sčítat nejen jednotlivé znaky, ale **celé textové řetězce**.

Příklad kódu pro Borland C++ 3.1

Zde je ukázka, jak naučit program provádět operace jako `text += "bb"`:

cpp

```
#include <iostream.h>
#include <string.h> // Potřebujeme pouze pro funkci strlen a strcpy

class MojeRetezec {
private:
    char data[100]; // Rezervujeme pevne místo pro vysledek (max 99 zr

public:
    // Konstruktor pro vytvoreni prazdneho retezce
    MojeRetezec() {
        data[0] = '\0';
    }

    // Konstruktor, který umožní zapsat: MojeRetezec text = "aa";
    MojeRetezec(const char* vychaziText) {
        strcpy(data, vychaziText);
    }

    // OPERÁTOR += pro pridani celeho retezce (napr. text += "bb")
    void operator+=(const char* pridavanyText) {
        // Kontrola, aby se text do pameti vesel
        if (strlen(data) + strlen(pridavanyText) < 100) {
            strcat(data, pridavanyText); // Spojí původní data s novyn
        }
    }

    // Metoda pro vypis textu
    const char* c_str() {
        return data;
    }
};

int main() {
    // 1. Vytvoreni objektu s vychozim textem "aa"
    MojeRetezec text = "aa";
    cout << "Puvodni text: " << text.c_str() << endl;

    // 2. Pridani textu pomoci operatoru +=
    text += "bb";
    cout << "Po prvnim pridani: " << text.c_str() << endl;

    // 3. Muzete pridavat opakovane
    text += "cc";
    text += "dd";
    cout << "Konecny vysledek: " << text.c_str() << endl;

    return 0;
}
```

Při použití kódu buďte obezřetní.

Zeptejte se na cokoli

+



proc nejde napsat primo "aa" += "bb" ?

Kdybyste napsal `"aa" += "bb"`, kompilátor vám vyhodí chybu *Lvalue required*. Pro něj jsou to jen dvě pevná místa v paměti (konstanty). Je to stejné, jako byste se pokusil v kódu napsat `5 += 3`. Číslo 5 také nemůžete změnit, musíte si vytvořit proměnnou `int cislo = 5;` a pak teprve udělat `cislo += 3;`.

Zajímá vás, jak k této třídě přidat i klasický **operátor** `+`, aby fungovalo řetězení typu `vysledek = text1 + text2` ?

Odpovědi od umělé inteligence můžou obsahovat chyby. [Další informace](#)

pokud se ale "strcat" da for cyklu tak blne ?

14:32

Ano, máte naprostou pravdu a je skvělé, že jste si toho všiml! Pokud dáte `strcat` do `for` cyklu (zejména pokud běží hodněkrát, např. 40x), začne program v Borland C++ buď „blbnout“ (generovat nesmyslné znaky), nebo rovnou v DOSu spadne (zamrzne).

Důvody, proč k tomu dochází, jsou dva: **přetečení paměti** a **princip, jakým strcat funguje**.

1. Proč strcat v cyklu selže (Princip fungování)

Funkce `strcat(A, B)` funguje tak, že:

1. Začne na začátku řetězce `A` a znak po znaku ho projíždí, dokud nenajde ukončovací znak `\0` (konec textu).
2. Teprve od tohoto místa začne kopírovat znaky z řetězce `B`.

Pokud v cyklu neustále voláte `strcat`, program musí při každém kroku procházet celý dosud vygenerovaný řetězec znovu od nuly. Co je ale horší: **pokud řetězec na začátku správně nevynulujete, strcat bloudí v paměti DOSu**, nenajde znak `\0`, začne přepisovat jiné části programu a Borland zkolabuje.

2. Jak to v cyklu opravit, aby to fungovalo na 100%

Pokud chcete v cyklu bezpečně spojovat řetězce nebo znaky bez použití složitých tříd, musíte udělat dvě věci:

- Na úplném začátku (před cyklem) musíte cílové pole **vynulovat** (vložit `\0` na první pozici).
- Musíte zajistit, aby pomocný připojovaný znak byl v paměti správně ukončen jako samostatný text.

Zde je ukázka, jak `strcat` v cyklu použít správně:

cpp

```
#include <iostream.h>
#include <string.h>

int main() {
    // 1. Vytvoříme dostatečně velké pole (rezerva pro výsledek)
    char vysledek[100];

    // !!! NEJDŮLEŽITĚJŠÍ KROK !!!
    // Nastavíme první znak na \0. Tím programu řekneme: "Tento řetězec
    // Pokud toto neuděláte, strcat v cyklu začne blbnout.
    vysledek[0] = '\0';
```

Zeptejte se na cokoli



Režim AI Vše Obrázky Vídeá Produkty Více ▾

```
// Teď už je strcat v cyklu bezpečný, protože ví, kde přesně ;
strcpy(vysledek, pomocny);
}

// Výpis výsledku (vypíše 40x 'X')
cout << "Výsledek v cyklu: " << vysledek << endl;

return 0;
}
```

Při použití kódu buďte obezřetní.

Rychlejší a bezpečnější alternativa pro cykly (Bez `strcpy`)

Vzhledem k tomu, že `strcpy` v každém kroku cyklu zbytečně prohledává už zapsané znaky, je v Borland C++ mnohem čistší a rychlejší **přístupovat k poli** **romo přes index** `[i]` a na `strcpy` se úplně vykašlat:

cpp

```
char vysledek[100];

for (int i = 0; i < 40; i++) {
    vysledek[i] = 'X'; // Znak zapíšeme rovnou na konkrétní pozici v p
}
vysledek[40] = '\0'; // Na úplný závěr (po skončení cyklu) text ručně
```

Při použití kódu buďte obezřetní.

Tento druhý způsob je v 16bitovém DOSu extrémně rychlý, spotřebovává minimum procesoru a nikdy nezpůsobí pád programu.

Chcete se podívat, jak tento indexový přístup zakomponovat do vašeho stávajícího kódu pro generování hesel, abyste měl jistotu, že už **nikdy** **blbnout** **nebude**?

Odpovědi od umělé inteligence mohou obsahovat chyby. [Další informace](#)

druha moznost se mi liby vic a funguje dobre. chi mit nekolik promennych a scitat je a take pridavat text v uvozovkach a ...

14:40

Při indexovém přístupu (druhá možnost) nemůžete pro sčítání použít operátor `+` nebo `+=` přímo mezi proměnnými, protože Borland C++ 3.1 je stále vnímá jako pouhá pole znaků.

Nejjednodušším, nejbezpečnějším a velmi přehledným řešením v C++ bez externích knihoven je použít funkci `sprintf` (z knihovny `<stdio.h>`) nebo nízkourovňové kopírování přes `strcpy` a `strcpy` (z knihovny `<string.h>`). Tyto funkce fungují v DOSu stoprocentně spolehlivě, pokud je cílové pole dostatečně velké.

Ukážu vám oba dva způsoby, jak sečíst několik proměnných, přidat k nim text v uvozovkách a vypsát výsledek.

Možnost A: Elegantní skládání pomocí `sprintf` (Doporučeno)

Zeptejte se na cokoli

+

Při použití kódu buďte obezřetní.

Vysledek: Zacatku: ABC12 - druhe: XYZ89 a treti: 456 (Konec)

Pokud chcete texty skládat postupně za sebe, musíte první proměnnou do cíle zkopírovat pomocí strcpy (tím se pole inicializuje a správně nastaví nulový znak) a všechny další texty už jen připojovat pomocí strcat.

Při použití kódu buďte obezřetní.

Vysledek: Heslo je: ABC12 a druhe je XYZ89!

U obou možností platí nejdůležitější pravidlo pro programování v DOSu: **Cílové**

+

 Režim AI Vše Obrázky Video Produkty Více ▾

  Přihlásit

=



Zeptejte se na cokoli

